C Strings — Worked Solutions

Part A answer key — approach, code, and why it's efficient

Solutions to the twelve string problems in the practice workbook. Each gives the **approach**, the **code**, the **why** (and its time/space cost), and the section of your **Strings guide** to read alongside it. Read the reasoning, then close this and rewrite each function from memory — that's where it sticks.

How to use this answer key

These are worked solutions to **Part A (Strings)** of the practice workbook. Each one gives the approach, the code, why that method is the efficient choice, and the section of your **Strings guide** to read alongside it.

> Study the *why*, then close this and rewrite the function from memory. Reconstructing it is the learning; copying it isn't.

Notation: **n** is the string length. Time/space is Big-O. "O(1) space" for the count-array solutions means constant (256 entries) — independent of n.

A1 — Reverse words in a sentence (M)

Approach: reverse the whole string, then reverse each word back.

```c
static void reverse(char *s, int i, int j) {
    while (i < j) {
        char t = s[i]; s[i] = s[j]; s[j] = t;
        i++; j--;
    }
}

void reverse_words(char *s) {
    int n = strlen(s);
    reverse(s, 0, n - 1);              // whole string
    int start = 0;
    for (int i = 0; i <= n; i++) {
        if (i == n || s[i] == ' ') {
            reverse(s, start, i - 1);  // one word
            start = i + 1;
        }
    }
}
```

Why / efficiency: `"the sky is blue"` reversed whole is `"eulb si yks eht"`; reversing each word fixes the letters, leaving the words in reverse order. It's all in-place swaps — **O(n) time, O(1) space** — so you never allocate a second buffer or build an array of word pointers. (Collapsing multiple spaces is an extra pass if you need it.)

Strings guide: §2 (memory — working in place) and the two-pointer idea.

A2 — Anagram check (M)

Approach: count each character of A, un-count each of B; equal iff all counts return to zero.

```c
bool is_anagram(const char *a, const char *b) {
    if (strlen(a) != strlen(b)) return false;
    int count[256] = {0};
    for (int i = 0; a[i]; i++) count[(unsigned char)a[i]]++;
    for (int i = 0; b[i]; i++) count[(unsigned char)b[i]]--;
    for (int i = 0; i < 256; i++) if (count[i]) return false;
    return true;
}
```

Why / efficiency: sorting both strings and comparing is O(n log n); counting is **O(n) time, O(1) space** (a fixed 256-entry table). The length pre-check rejects mismatches instantly. The `(unsigned char)` cast keeps negative `char` values from indexing out of bounds.

Strings guide: §3 (length) and §8 (the `(unsigned char)` cast).

A3 — First non-repeating character (M)

Approach: one pass to count frequencies, a second to return the first char with count 1.

```c
int first_unique(const char *s) {
    int count[256] = {0};
    for (int i = 0; s[i]; i++) count[(unsigned char)s[i]]++;
    for (int i = 0; s[i]; i++)
        if (count[(unsigned char)s[i]] == 1) return i;
    return -1;                          // none found
}
```

Why / efficiency: the naive "for each char, scan the rest" is O(n²). Two linear passes with a count table are **O(n) time, O(1) space**. Returning the index (not the char) lets the caller handle "not found" with `-1`.

Strings guide: §3 (walking to `'\0'`).

A4 — Run-length encode (M)

Approach: walk the string; for each run of equal characters, emit the char then the run length. Write with `snprintf` so you can't overflow the output.

```c
// "aaabbc" -> "a3b2c1"; returns length, or -1 if dst too small
int rle_encode(const char *src, char *dst, size_t dstsize) {
    size_t out = 0;
    for (size_t i = 0; src[i]; ) {
        char c = src[i];
        size_t run = 0;
        while (src[i] == c) { i++; run++; }
        int n = snprintf(dst + out, dstsize - out, "%c%zu", c, run);
        if (n < 0 || (size_t)n >= dstsize - out) return -1;  // truncated
        out += n;
    }
    return (int)out;
}
```

Why / efficiency: a single pass, **O(n) time**. Using `snprintf` with the *remaining* buffer size on each write means a too-small `dst` is detected, not overrun. (Decode is the mirror: read a char, read its number, emit that many copies.)

Strings guide: §6 (`snprintf` as the safe writer).

A5 — Implement `atoi` (H)

Approach: skip spaces, read an optional sign, accumulate digits, and clamp on overflow *before* it happens.

```c
int my_atoi(const char *s) {
    int i = 0, sign = 1, result = 0;
    while (isspace((unsigned char)s[i])) i++;
    if (s[i] == '+' || s[i] == '-') {
        if (s[i] == '-') sign = -1;
        i++;
    }
    while (isdigit((unsigned char)s[i])) {
        int d = s[i] - '0';
        if (result > INT_MAX / 10 ||
            (result == INT_MAX / 10 && d > INT_MAX % 10))
            return sign == 1 ? INT_MAX : INT_MIN;
        result = result * 10 + d;
        i++;
    }
    return sign * result;
}
```

Why / efficiency: O(n) time, O(1) space. The key is the overflow check: signed overflow is undefined behavior, so you must test *before* multiplying — if `result` already exceeds `INT_MAX/10`, or equals it with a final digit past `INT_MAX%10`, clamp. This is the detail interviewers look for.

Strings guide: §3 (char walking), §8 (`isspace` / `isdigit`).

A6 — Implement `itoa` (H)

Approach: extract digits with `% 10` / `/ 10` (which produces them reversed), add the sign, then reverse the buffer.

```c
void my_itoa(int value, char *buf) {
    int i = 0, negative = (value < 0);
    unsigned int n = negative ? (0u - (unsigned)value)
                              : (unsigned)value;
    do {                                // at least one digit (handles 0)
        buf[i++] = '0' + (n % 10);
        n /= 10;
    } while (n > 0);
    if (negative) buf[i++] = '-';
    buf[i] = '\0';
    for (int a = 0, b = i - 1; a < b; a++, b--) {   // reverse in place
        char t = buf[a]; buf[a] = buf[b]; buf[b] = t;
    }
}
```

Why / efficiency: O(d) time (d = digit count). The `do/while` guarantees `0` prints as `"0"`. Converting to `unsigned` with `0u - (unsigned)value` gives the correct magnitude even for `INT_MIN`, whose positive value doesn't fit in `int` — negating it directly would be undefined behavior.

Strings guide: §2 (caller buffer) and §4 (the in-place reversal pattern).

A7 — Palindrome, ignoring case and punctuation (M)

Approach: two pointers from the ends; skip non-alphanumerics; compare lowercased.

```c
bool is_palindrome(const char *s) {
    int i = 0, j = (int)strlen(s) - 1;
    while (i < j) {
        while (i < j && !isalnum((unsigned char)s[i])) i++;
        while (i < j && !isalnum((unsigned char)s[j])) j--;
        if (tolower((unsigned char)s[i]) != tolower((unsigned char)s[j]))
            return false;
        i++; j--;
    }
    return true;
}
```

Why / efficiency: O(n) time, O(1) space. The two-pointer scan skips junk in place, so you never build a cleaned-up copy of the string first — one pass, no allocation.

Strings guide: §8 (`isalnum` / `tolower`), two-pointer.

A8 — Longest substring without repeating characters (H)

Approach: sliding window. Keep the last index each character was seen; when a character repeats *inside* the window, jump the window's start past its previous position.

```c
int longest_unique(const char *s) {
    int last[256];
    memset(last, -1, sizeof(last));     // -1 = "not seen yet"
    int start = 0, best = 0;
    for (int i = 0; s[i]; i++) {
        unsigned char c = s[i];
        if (last[c] >= start) start = last[c] + 1;   // shrink window
        last[c] = i;
        int len = i - start + 1;
        if (len > best) best = len;
    }
    return best;
}
```

Why / efficiency: brute force checks every substring in O(n²) (or worse). The window advances each pointer at most once, with O(1) lookups in the `last` table — **O(n) time, O(1) space**. `memset(last, -1, ...)` works because all-bits-1 is `-1` for `int`, a clean way to fill the table.

Strings guide: §3 (walking) and the memory functions (`memset`).

A9 — Your own `strtok` (reentrant) (H)

Approach: mirror `strtok_r` — skip leading delimiters, mark the token, scan to the next delimiter, terminate it, and store the resume position in a caller-owned `*saveptr` (no hidden global).

```c
char *my_strtok(char *str, const char *delims, char **saveptr) {
    char *s = str ? str : *saveptr;
    if (!s) return NULL;
    s += strspn(s, delims);             // skip leading delimiters
    if (*s == '\0') { *saveptr = NULL; return NULL; }
    char *token = s;
    s += strcspn(s, delims);            // go to next delimiter
    if (*s) { *s = '\0'; *saveptr = s + 1; }
    else    { *saveptr = NULL; }
    return token;
}
```

Why / efficiency: across all calls it touches each character a constant number of times — **O(n) total**. Passing `saveptr` instead of using a global makes it *reentrant*: safe in threads and nested loops, which the real `strtok` is not. `strspn` / `strcspn` do the delimiter scanning for you.

Strings guide: §7 (`strspn` / `strcspn`) and the note on `strtok`'s design flaw.

A10 — All permutations of a string (H)

Approach: backtracking — fix position `start` by swapping each later character into it, recurse on the rest, then swap back.

```c
static void swap(char *a, char *b) { char t = *a; *a = *b; *b = t; }

void permute(char *s, int start) {
    if (s[start] == '\0') { printf("%s\n", s); return; }
    for (int i = start; s[i]; i++) {
        swap(&s[start], &s[i]);
        permute(s, start + 1);
        swap(&s[start], &s[i]);         // backtrack
    }
}
```

Why / efficiency: there are n! permutations and each takes O(n) to print, so **O(n · n!) time** — optimal, since you must emit that many. Space is **O(n)** for the recursion depth, and it permutes the string in place rather than building new strings. (Repeated input characters produce duplicates; skipping those is the stretch.)

Strings guide: §2 (in-place) plus recursion.

A11 — Count words (M)

Approach: a state machine — track whether you're currently inside a word, and count each transition from "outside" to "inside."

```c
int count_words(const char *s) {
    int count = 0, in_word = 0;
    for (int i = 0; s[i]; i++) {
        if (isspace((unsigned char)s[i])) in_word = 0;
        else if (!in_word) { in_word = 1; count++; }  // word start
    }
    return count;
}
```

Why / efficiency: O(n) time, O(1) space. Counting only the "entering a word" transitions handles multiple, leading, and trailing spaces for free — no tokenizing and no allocation, unlike a split-into-array approach.

Strings guide: §3 (walking), §8 (`isspace`).

A12 — Wildcard match (`?` and `*`) (H)

Approach: greedy two-pointer. Advance on a literal or `?` match; on `*`, remember its position and the current text spot; on a mismatch, backtrack to just after the last `*` and consume one more text character.

```c
bool wildcard(const char *s, const char *p) {
    const char *star = NULL, *ss = NULL;
    while (*s) {
        if (*p == '?' || *p == *s) { s++; p++; }
        else if (*p == '*') { star = p++; ss = s; }   // remember '*'
        else if (star) { p = star + 1; s = ++ss; }    // backtrack
        else return false;
    }
    while (*p == '*') p++;              // trailing '*' matches empty
    return *p == '\0';
}
```

Why / efficiency: the DP solution is O(n·m) time *and* O(n·m) space; this greedy version keeps the same worst-case time but only **O(1) space**, because it stores just the last `*` position instead of a full table. The trailing-`*` loop lets a `*` at the end match the empty remainder.

Strings guide: §2 (pointers) and string walking.